A Voyage to the Kernel

Part 22

Segment: 4.2, Day 21

In the previous article, we'd looked at how to add more features to a module. But you couldn't call what we'd created a 'device driver'. In this article, we will devote our time to developing a driver for a simple 'device'. As we discussed before, Linux treats devices as files. You may scan through your */dev/* directory to see the listing. It is through the addition of a file node in this directory that we make a physical device accessible.

You may notice that many files are listed in this directory. This does not mean that all these files (devices) are active. Issue the following command to see the active ones:

```
cat /proc/devices
```

We have also seen that there are character and block devices. (We looked at network devices as well.) The type of the device informs us about the way in which data will be written to the corresponding device. In the case of a character device, it is added serially (byte by byte) and for a block device, the data is added as large segments (an ideal example will be HDD).

Now I am going to list the active devices in my system:

```
aasisvinayak@GNU-BOX:~$ cat /proc/devices
Character devices:
  1 mem
  4 /dev/vc/0
  4 tty
  4 ttyS
  5 /dev/tty
  5 /dev/console
  5 /dev/ptmx
  6 lp
  7 vcs
 10 misc
 13 input
 14 sound
 21 sg
 29 fb
 81 video4linux
 99 ppdev
108 ppp
116 alsa
128 ptm
136 pts
180 usb
189 usb_device

<<<OUTPUT TRUNCATED>>>
```

Here I can see some names and a numerical value. The numeral is actually the major number associated with the device driver. (We will discuss major and minor numbers after writing the code for the new driver.)

Having got this information, I can now allocate a major number that has not been assigned previously. All I need to note is that the new number should not be on this list.

Now let me show you the complete code for the new driver:

```
#include"LFY_device_header.h"
#include<linux/module.h>
#include<linux/init.h>

MODULE_AUTHOR("Aasis Vinayak PG");
MODULE_DESCRIPTION("Written for Voyage to Kernel");

static int LFY_device_init(void);
static void LFY_device_cleanup(void);

module_init(LFY_device_init);
module_exit(LFY_device_cleanup);

static int LFY_device_init(void)
{
if(register_chrdev(161, "LFY_device",&LFY_ops))
{
      printk("<1>failed to register");
}
return 0;
```